Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Laboratorio de Organización Computacional

Sección C



SmartSort: Clasificadora de paquetes

Fecha 24/10/2026

Grupo 7	
Nombre	**Carnet**
Brandon Otoniel Poitán Gálvez	**202300404**
Luis Fernando Orive Perez	**202100262**
Brandón Adán González Oliva	**202307694**
Marcos Aarón Toledo Álvarez	**202200013**
Douglas Xavier Santiago Soto Mejía	**202201502**

Indice

Contenido

Indice 2
Introduccion 3
Objetivos 3
Codigo Arduino 3
Proteus 15
Maqueta 18
Presupuesto 28
Aportes 29
Equipo utilizado 29
Conclusiones 32

Introduccion

SmartSort es un sistema automatizado diseñado para clasificar paquetes según su tamaño, utilizando sensores láser, un Arduino y servomotores para dirigirlos hacia el contenedor correspondiente. Además, el sistema registra estadísticas de producción en la memoria EEPROM y permite el control remoto mediante Bluetooth, integrando diferentes componentes electrónicos para representar un proceso de clasificación industrial.

Objetivos

General

Desarrollar un prototipo funcional de una clasificadora automática de paquetes capaz de detectar, clasificar y desviar paquetes según su tamaño, registrando las estadísticas de producción y permitiendo el control del sistema mediante Bluetooth.

Especifico

1. Implementar un sistema de detección mediante sensores láser y LDR que permita identificar correctamente las diferentes categorías de paquetes.
2. Controlar la banda transportadora y los servomotores mediante Arduino para realizar el desvío de los paquetes hacia el contenedor correspondiente.
3. Registrar y gestionar las estadísticas de clasificación en la memoria EEPROM y permitir el monitoreo y control del sistema mediante Bluetooth y una pantalla LCD. 0964_Proyecto_1_2S2026

Codigo Arduino

```
/* SmartSort rele D5 - Mega 2560 + 3 LDR + control digital activo HIGH.
 * Revisar Configuracion.h y GUIA_CONEXIONES.md antes de energizar.
 * Circuito externo probado por el usuario: HIGH arranca, LOW para. Un paquete a la vez.
 * Recorrido estimado por tiempo de motor ON: no hay encoder.
 */
#include <Arduino.h>
#if !defined(__AVR_ATmega2560__)
#error "Seleccione Arduino Mega 2560 (ATmega2560). Este mapa no es para Uno/ESP32."
#endif
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
#include <EEPROM.h>
#include <stdint.h>
#include <stddef.h>
#include <ctype.h>
#include <stdlib.h>
#include <string.h>

#include "Configuracion.h"

LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Servo s1, s2;
uint32_t tiempoMotor = 0, relojMotor = 0;
bool motorActivado = false;
bool monitorSensores = false;

#include "Configuracion.h"

LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Servo s1, s2;
uint32_t tiempoMotor = 0, relojMotor = 0;
bool motorActivado = false;
bool monitorSensores = false;
uint32_t ultimoMonitor = 0, libreDesde = 0;
bool portalLibre = false;
bool lcdOK = false, eepromFallo = false, operacionFallo = false;
bool marcha = false, pausado = false, exigirVacia = false;
enum Modo : uint8_t { NORMAL, LOTE, MANTENIMIENTO };
enum Tipo : uint8_t { NINGUNO, GRANDE, PEQUENO, RECHAZO };
enum Fase : uint8_t { LIBRE, LEYENDO, TRANSPORTANDO };
Modo modo = NORMAL;
Tipo tipo = NINGUNO, ultimo = NINGUNO;
Fase fase = LIBRE;
uint32_t loteObjetivo = 0, loteHechos = 0, origen = 0;

bool vioBajo = false, vioAlto = false;
bool bloqueadoBajo = false, bloqueadoAlto = false;
bool presenciaCruda = false, presenciaEstable = false;
uint32_t cambioPresencia = 0;
uint32_t ledHasta = 0;
```

```
bool bloqueadoBajo = false, bloqueadoAlto = false;
bool presenciaCruda = false, presenciaEstable = false;
uint32_t cambioPresencia = 0;
uint32_t ledHasta = 0;
bool ledEvento = false;
uint8_t beepsPendientes = 0;
bool sonando = false;
bool tonoLotePendiente = false;
uint32_t sonidoCambio = 0;
uint16_t sonidoDuracion = 100, sonidoFrecuencia = 1800;
bool reporteVisible = false;
uint32_t reporteCambio = 0;
uint8_t paginaReporte = 0;

// Registros alternados. Ver tabla exacta en GUIA_CONEXIONES.md.
struct __attribute__((packed)) Registro {
  uint16_t magic;
  uint8_t version;
  uint32_t secuencia;
  uint32_t grandes, pequenos, rechazos, historico;
  uint16_t crc;
  uint8_t commit;
};
static_assert(sizeof(Registro) == 26, "Formato EEPROM inesperado");

,,
static_assert(sizeof(Registro) == 26, "Formato EEPROM inesperado");
Registro datos = {};
int ranuraActual = -1;
const int DIRECCIONES[2] = {0, 64};

// Declaraciones explicitas para tipos propios en el preprocesador Arduino.
const char *nombreTipo(Tipo t);
uint16_t crcRegistro(const Registro &r);
bool valido(const Registro &r);
bool guardar(Registro nuevo);
void fallo(const char *causa, bool memoria = false);
struct Entrada { char texto[48]; uint8_t n; bool exceso; };
void recibir(Stream &puerto, Entrada &e);

void actualizarRelojMotor() {
  uint32_t ahora = millis();
  if (motorActivado) tiempoMotor += ahora - relojMotor;
  relojMotor = ahora;
}

uint32_t leerTiempoMotor() {
```

```
uint32_t leerTiempoMotor() {
  actualizarRelojMotor();
  return tiempoMotor;
}

void motor(bool activar) {
  actualizarRelojMotor();
  motorActivado = activar && POTENCIA_VERIFICADA;
  digitalWrite(PIN_SENAL, motorActivado ? HIGH : LOW);
}

void decir(const char *texto) {
  Serial.println(texto);
  Serial1.println(texto);
}

void fila(uint8_t n, const char *texto) {
  if (!lcdOK) return;
  lcd.setCursor(0, n);
  bool fin = false;
  for (uint8_t i = 0; i < 16; ++i) {
    char c = ' ';
```

```
  if (eepromFallo || operacionFallo) decir("FALLO: revisar antes de reanud
}

uint16_t crcRegistro(const Registro &r) {
  const uint8_t *p = (const uint8_t *)&r;
  uint16_t crc = 0xFFFF;
  for (uint8_t i = 0; i < offsetof(Registro, crc); ++i) {
    crc ^= (uint16_t)p[i] << 8;
    for (uint8_t b = 0; b < 8; ++b)
      crc = (crc & 0x8000) ? (uint16_t)((crc << 1) ^ 0x1021) :
                             (uint16_t)(crc << 1);
  }
  return crc;
}

bool valido(const Registro &r) {
  uint64_t turno = (uint64_t)r.grandes + r.pequenos + r.rechazos;
  return r.commit == 0xA5 && r.magic == 0x5353 && r.version == 1 &&
         r.crc == crcRegistro(r) && turno <= r.historico;
}

void fallo(const char *causa, bool memoria) {
  marcha = false;
  motor(false);
```

```
 EEPROM.update(base + offsetof(Registro, commit), 0);
 const uint8_t *p = (const uint8_t *)&nuevo;
 for (uint8_t i = 0; i < offsetof(Registro, commit); ++i)
   EEPROM.update(base + i, p[i]);
 EEPROM.update(base + offsetof(Registro, commit), 0xA5);
 Registro comprobacion;
 EEPROM.get(base, comprobacion);
 if (!valido(comprobacion) || memcmp(&nuevo, &comprobacion, sizeof(nuevo))) {
   fallo("No se pudo verificar escritura EEPROM.", true); return false;
 }
 datos = nuevo; ranuraActual = destino;
 return true;


oid cargarEEPROM() {
 Registro a, b;
 EEPROM.get(0, a); EEPROM.get(64, b);
 bool va = valido(a), vb = valido(b);
 if (va || vb) {
   ranuraActual = !va ? 1 : !vb ? 0 :
     ((int32_t)(b.secuencia - a.secuencia) > 0 ? 1 : 0);
   datos = ranuraActual == 0 ? a : b;
   return;
 }
 bool virgen = true;
 for (uint8_t s = 0; s < 2; ++s)
   for (uint8_t i = 0; i < sizeof(Registro); ++i)
     if (EEPROM.read(DIRECCIONES[s] + i) != 0xFF) virgen = false;
 if (virgen) guardar(datos);
 else fallo("EEPROM sin registro valido: no se borra automaticamente.", true);
```

```
  if (paginaReporte == 0) {
    snprintf(a, sizeof(a), "G:%lu", (unsigned long)datos.grandes);
    snprintf(b, sizeof(b), "P:%lu", (unsigned long)datos.pequenos);
  } else {
    snprintf(a, sizeof(a), "R:%lu", (unsigned long)datos.rechazos);
    snprintf(b, sizeof(b), "Total:%lu", (unsigned long)datos.historico);
  }
  fila(0, a); fila(1, b);
}

void reporte() {
  char texto[100];
  snprintf(texto, sizeof(texto), "G:%lu P:%lu R:%lu Historico:%lu",
    (unsigned long)datos.grandes, (unsigned long)datos.pequenos,
    (unsigned long)datos.rechazos, (unsigned long)datos.historico);
  decir(texto);
  paginaReporte = 0; reporteVisible = true; reporteCambio = millis();
  paginaEstadisticas();
}

void sonido(uint8_t cantidad, uint16_t hz, uint16_t duracion) {
  noTone(BUZZER); sonando = false;
  beepsPendientes = cantidad; sonidoFrecuencia = hz;
  sonidoDuracion = duracion; sonidoCambio = millis();
}

void actualizarAvisos() {
  uint32_t ahora = millis();
  digitalWrite(AZUL, HIGH); // encendido = controlador listo para comandos
  bool activo = ledEvento && (int32_t)(ledHasta - ahora) > 0;
  digitalWrite(VERDE, activo && ultimo != RECHAZO);
  digitalWrite(ROJO, eepromFallo || operacionFallo || (activo && ultimo == RECHAZO));
  if (sonando && ahora - sonidoCambio >= sonidoDuracion) {
```

```
    --beepsPendientes; sonidoCambio = ahora;
  }
  if (!sonando && !beepsPendientes && tonoLotePendiente) {
    tonoLotePendiente = false;
    sonido(1, 1400, 1200);
  }
  if (reporteVisible && ahora - reporteCambio >= 2000) {
    paginaReporte ^= 1; paginaEstadisticas(); reporteCambio = ahora;
  }
}

bool actualizarLDR(uint8_t pin, uint16_t umbral, bool anterior) {
  int v = analogRead(pin);
  if (v < (int)umbral - HISTERESIS) return true;
  if (v > (int)umbral + HISTERESIS) return false;
  return anterior;
}

void terminarPaquete() {
  // Mantiene la senal durante el guardado: no conmuta el rele por paquete.
  // EEPROM es sincrona: los comandos pueden esperar unas decenas de ms.
  Registro nuevo = datos;
  uint32_t contador = tipo == GRANDE ? nuevo.grandes :
                      tipo == PEQUENO ? nuevo.pequenos : nuevo.rechazos;
  if (nuevo.historico == UINT32_MAX || contador == UINT32_MAX) {
    fallo("Limite de contadores: paquete sin registrar.", true); return;
  }
  if (tipo == GRANDE) ++nuevo.grandes;
  else if (tipo == PEQUENO) ++nuevo.pequenos;
  else ++nuevo.rechazos;
  ++nuevo.historico;
  if (!guardar(nuevo)) return;
  ultimo = tipo; tipo = NINGUNO; fase = LIBRE;
```

```
  if (modo == LOTE && ++loteHechos >= loteObjetivo) {
    marcha = false; pausado = false; motor(false);
    tonoLotePendiente = true; // conserva primero los beeps del ultimo paquete
    decir("Lote completo"); reporte();
    mostrar("Lote completo", "reporte: ver datos");
    // El informe completo tambien ya fue enviado al telefono.
    // LCD vuelve automaticamente al reporte tras el mensaje.
    reporteVisible = true; reporteCambio = millis(); paginaReporte = 1;
  } else {
    motor(true);
    decir("Listo para el siguiente paquete.");
  }
}

void actualizarProceso() {
  bloqueadoBajo = actualizarLDR(LDR_BAJO, UMBRAL_BAJO, bloqueadoBajo);
  bloqueadoAlto = actualizarLDR(LDR_ALTO, UMBRAL_ALTO, bloqueadoAlto);
  uint32_t ahora = millis();
  bool cruda = actualizarLDR(LDR_PRESENCIA, UMBRAL_PRESENCIA, presenciaCruda);
  if (cruda != presenciaCruda) { presenciaCruda = cruda; cambioPresencia = ahora; }
  bool flanco = false;
  if (ahora - cambioPresencia >= FILTRO_PRESENCIA_MS && presenciaEstable != presenciaCruda) {
    presenciaEstable = presenciaCruda; flanco = presenciaEstable;
  }
  bool libre = !presenciaCruda && !presenciaEstable && !bloqueadoBajo && !bloqueadoAlto;
  if (!libre) { libreDesde = ahora; portalLibre = false; }
  else portalLibre = ahora - libreDesde >= PORTAL_LIBRE_MS;
  if (!marcha || eepromFallo || operacionFallo || modo == MANTENIMIENTO) return;
  if (flanco) {
    if (fase != LIBRE) {
      fallo("Segundo paquete demasiado cerca. Retirar y cancelar proceso."); return;
    }
    origen = leerTiempoMotor();
```

```
      else if (viaBaja)
        tipo = LOGICA_LITERAL_DOCUMENTO ? GRANDE : PEQUENO;
      else tipo = RECHAZO;
      fase = TRANSPORTANDO;
      char texto[32];
      snprintf(texto, sizeof(texto), "Paq: %s", nombreTipo(tipo));
      mostrar(texto, "En transporte"); decir(texto);
    }
  }
  if (fase == TRANSPORTANDO) {
    uint32_t recorrido = leerTiempoMotor() - origen;
    s1.write(tipo == GRANDE && recorrido >= S1_DESDE_MS && recorrido < S1_HASTA_MS ? S1_ACTIVO : S1_NEU
    s2.write(tipo == PEQUENO && recorrido >= S2_DESDE_MS && recorrido < S2_HASTA_MS ? S2_ACTIVO : S2_NE
    if (recorrido >= SALIDA_MS) {
      if (presenciaEstable) { fallo("Portal sigue bloqueado. Revisar atasco."); return; }
      terminarPaquete();
    }
  }
}

bool puedeArrancar() {
  if (!POTENCIA_VERIFICADA) { decir("BLOQUEADO: verificar motor, fuente y circuito de control."); retur
  if (!SERVOS_VERIFICADOS) { decir("BLOQUEADO: verificar alimentacion y posicion de servos."); return f
  if (modo != MANTENIMIENTO && !CALIBRACION_LISTA) { decir("BLOQUEADO: calibrar LDR y tiempos antes de
  if (!lcdOK) { decir("BLOQUEADO: LCD sin respuesta al arrancar."); return false; }
  if (eepromFallo || operacionFallo) { decir("BLOQUEADO: fallo activo."); return false; }
  if (exigirVacia) { decir("Retirar paquetes y enviar confirmar_vacia."); return false; }
  if (modo != MANTENIMIENTO && fase == LIBRE && !portalLibre) {
    decir("Despeje los tres haces antes de iniciar."); return false;
  }
  if (modo == LOTE && loteHechos >= loteObjetivo) {
    decir("Lote terminado: configurar otro lote o modo_normal."); return false;
  }
  return true;
```

```
bool numeroLote(const char *p, uint32_t &n) {
  if (!*p) return false;
  n = 0;
  for (; *p; ++p) {
    if (*p < '0' || *p > '9') return false;
    uint8_t d = *p - '0';
    if (n > (10000UL - d) / 10) return false;
    n = n * 10 + d;
  }
  return n >= 1 && n <= 10000;
}

void mostrarSensores() {
  int a0 = analogRead(LDR_BAJO), a1 = analogRead(LDR_ALTO), a2 = analogRead(LDR_PRESENCIA);
  char b[64], l1[32], l2[32];
  snprintf(b, sizeof(b), "A0:%d A1:%d A2:%d presencia:%d", a0, a1, a2, presenciaEstable);
  decir(b);
  snprintf(l1, sizeof(l1), "A0:%d A1:%d", a0, a1);
  snprintf(l2, sizeof(l2), "A2:%d P:%d", a2, presenciaEstable);
  mostrar(l1, l2);
}

void comando(char *p) {
  while (*p == ' ') ++p;
  size_t len = strlen(p);
  while (len && p[len - 1] == ' ') p[--len] = 0;
  for (char *q = p; *q; ++q) *q = tolower((unsigned char)*q);
  if (!strcmp(p, "estado")) { estado(); return; }
  if (!strcmp(p, "reporte")) { reporte(); return; }
  if (!strcmp(p, "sensores")) { mostrarSensores(); return; }
```

```
}
if (!strcmp(p, "neutros")) {
  if (marcha || fase != LIBRE || !SERVOS_VERIFICADOS) { decir("Detener y verificar servos."); return;
  s1.write(S1_NEUTRO); s2.write(S2_NEUTRO); decir("Servos neutrales."); return;
}
if (!strcmp(p, "iniciar")) {
  if (puedeArrancar()) { marcha = true; pausado = false; motor(true); estado(); }
  return;
}
if (!strcmp(p, "pausa") || !strcmp(p, "pausar")) {
  marcha = false; pausado = true; motor(false);
  mostrar("PAUSADO", "Banda: OFF"); decir("PAUSADO: iniciar reanuda el recorrido."); return;
}
if (!strcmp(p, "detener")) {
  marcha = false; pausado = false; motor(false);
  s1.write(S1_NEUTRO); s2.write(S2_NEUTRO);
  fase = LIBRE; tipo = NINGUNO; exigirVacia = true; modo = NORMAL;
  if (!eepromFallo) operacionFallo = false;
  reporte(); decir("DETENIDO: retire TODOS los paquetes y envie confirmar_vacia."); return;
}
if (!strcmp(p, "confirmar_vacia")) {
  if (marcha || !portalLibre) { decir("Detenga y despeje el portal."); return; }
  exigirVacia = false; decir("Banda vacia confirmada por operario."); return;
}
if (!strcmp(p, "resetear")) {
  if (marcha || fase != LIBRE || eepromFallo || modo == MANTENIMIENTO) { decir("Detenga y retire paqu
  Registro nuevo = datos; nuevo.grandes = nuevo.pequenos = nuevo.rechazos = 0;
  if (guardar(nuevo)) {
    loteHechos = 0; modo = NORMAL;
    mostrar("Reset OK", "Nuevo turno listo"); decir("Reset OK; historico preservado.");
  }
  return;
```

```
  if (p[1] == '1') s1.write((uint8_t)angulo);
  else s2.write((uint8_t)angulo);
  decir("Angulo de prueba enviado."); return;
}
if (!strcmp(p, "neutros")) {
  if (marcha || fase != LIBRE || !SERVOS_VERIFICADOS) { decir("Detener y verificar servos."); return; }
  s1.write(S1_NEUTRO); s2.write(S2_NEUTRO); decir("Servos neutrales."); return;
}
if (!strcmp(p, "iniciar")) {
  if (puedeArrancar()) { marcha = true; pausado = false; motor(true); estado(); }
  return;
}
if (!strcmp(p, "pausa") || !strcmp(p, "pausar")) {
  marcha = false; pausado = true; motor(false);
  mostrar("PAUSADO", "Banda: OFF"); decir("PAUSADO: iniciar reanuda el recorrido."); return;
}
if (!strcmp(p, "detener")) {
  marcha = false; pausado = false; motor(false);
  s1.write(S1_NEUTRO); s2.write(S2_NEUTRO);
  fase = LIBRE; tipo = NINGUNO; exigirVacia = true; modo = NORMAL;
  if (!eepromFallo) operacionFallo = false;
  reporte(); decir("DETENIDO: retire TODOS los paquetes y envie confirmar_vacia."); return;
}
if (!strcmp(p, "confirmar_vacia")) {
  if (marcha || !portalLibre) { decir("Detenga y despeje el portal."); return; }
  exigirVacia = false; decir("Banda vacia confirmada por operario."); return;
}
if (!strcmp(p, "resetear")) {
  if (marcha || fase != LIBRE || eepromFallo || modo == MANTENIMIENTO) { decir("Detenga y retire paquetes antes de resetear."); return; }
  Registro nuevo = datos; nuevo.grandes = nuevo.pequenos = nuevo.rechazos = 0;
  if (guardar(nuevo)) {
    loteHechos = 0; modo = NORMAL;
    mostrar("Reset OK", "Nuevo turno listo"); decir("Reset OK; historico preservado.");
  }
```

```
    if (!strcmp(p, "modo_normal")) modo = NORMAL;
    else if (!strcmp(p, "modo_mantenimiento")) modo = MANTENIMIENTO;
    else {
      uint32_t n;
      if (!numeroLote(p + 10, n)) { mostrar("ERROR: lote", "Usar 1 a 10000"); decir("ERROR: lote entero de 1 a 10000."); return; }
      modo = LOTE; loteObjetivo = n; loteHechos = 0;
    }
    if (!puedeArrancar()) { modo = anterior; loteObjetivo = objetivoAnterior; loteHechos = hechosAnteriores; return; }
    s1.write(S1_NEUTRO); s2.write(S2_NEUTRO);
    marcha = true; pausado = false; motor(true); estado();
    if (modo == LOTE) {
      char b[32]; snprintf(b, sizeof(b), "%lu paquetes", (unsigned long)loteObjetivo);
      mostrar("Lote activo", b);
    }
    return;
  }
  mostrar("ERROR: Cmd", "invalido"); decir("ERROR: Cmd invalido");
}

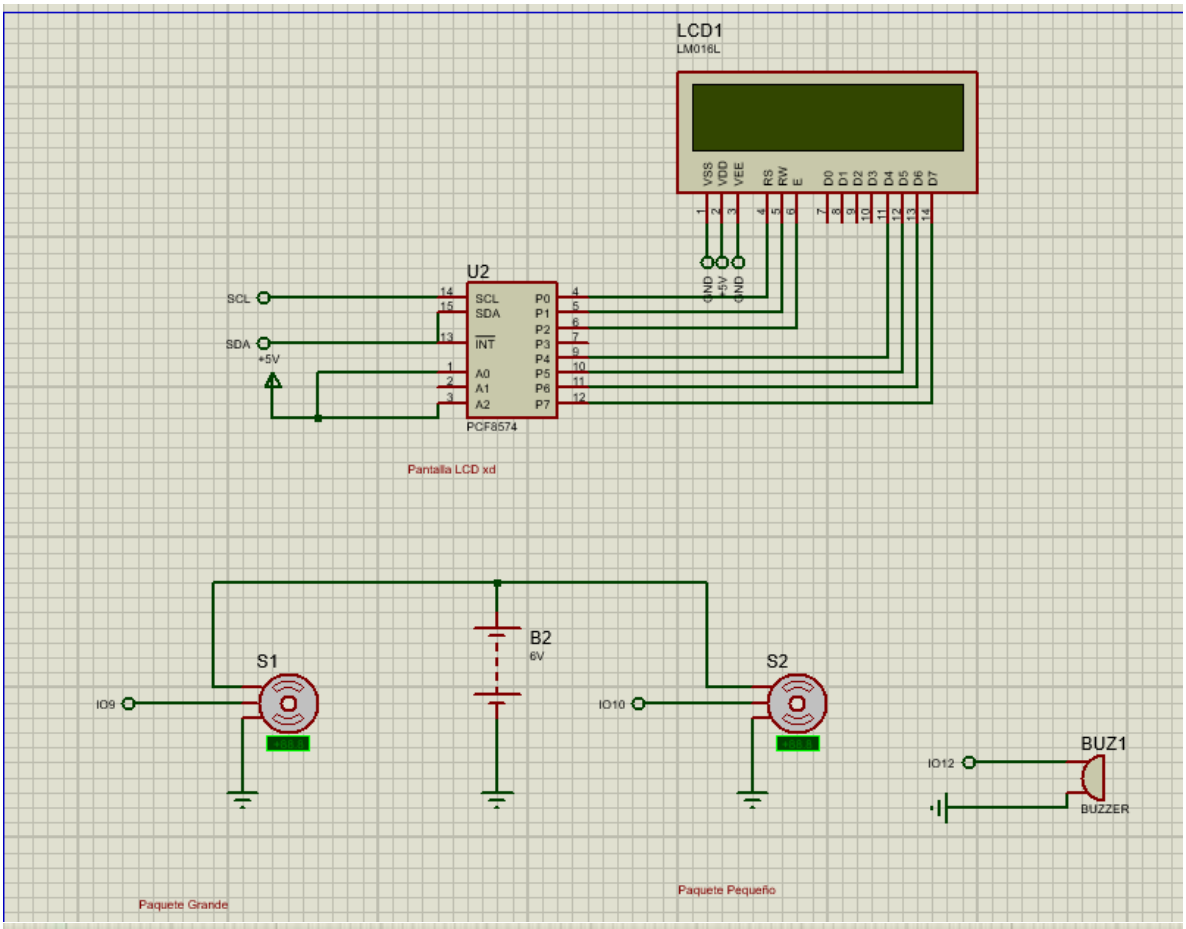
Entrada usb = {}, bt = {};
void recibir(Stream &puerto, Entrada &e) {
  for (uint8_t presupuesto = 0; presupuesto < 32 && puerto.available(); ++presupuesto) {
    char c = puerto.read();
    if (c == '\r' || c == '\n') {
      if (e.exceso) { mostrar("ERROR: Cmd", "muy largo"); decir("ERROR: comando demasiado largo."); }
      else if (e.n) { e.texto[e.n] = 0; comando(e.texto); }
      e.n = 0; e.exceso = false;
    } else if (!e.exceso) {
      if (e.n < sizeof(e.texto) - 1) e.texto[e.n++] = c;
      else e.exceso = true;
    }
  }
}

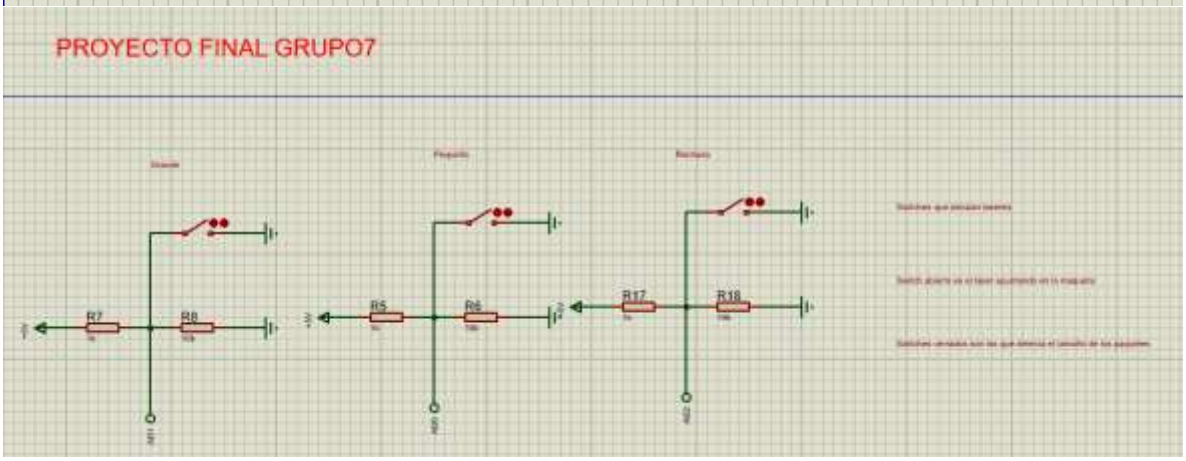
void setup() {
```

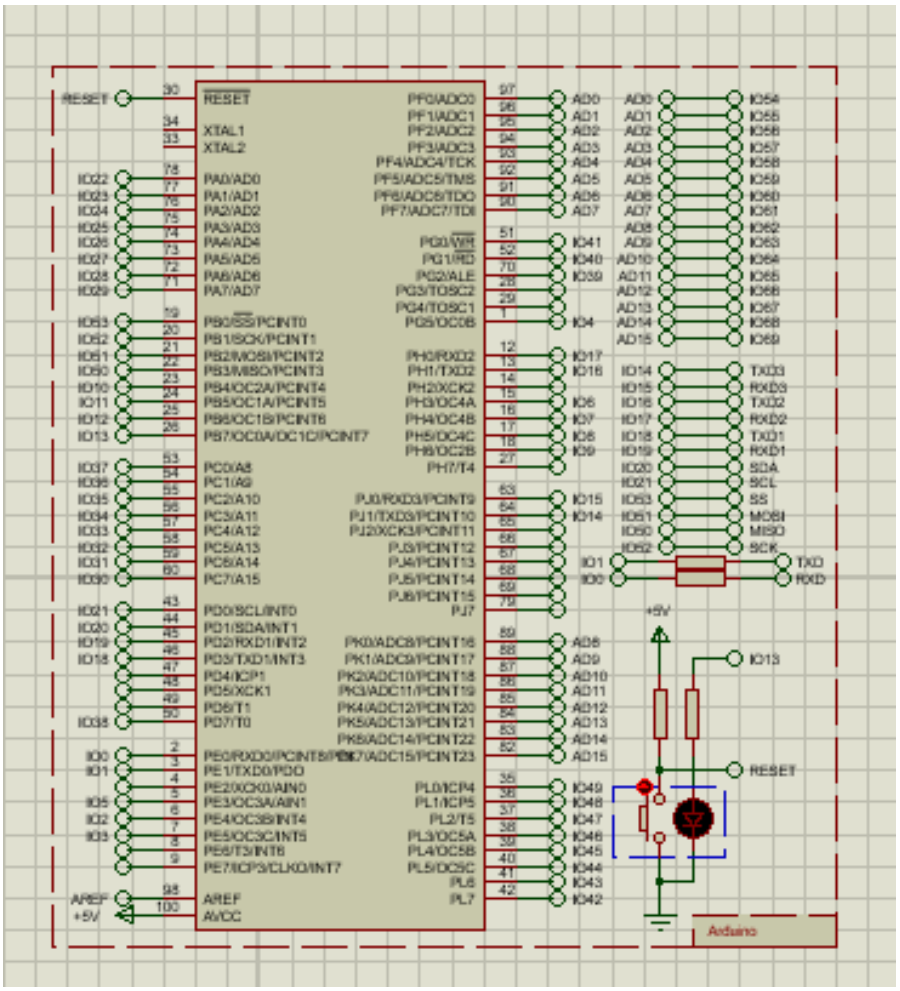
```
void setup() {
  digitalWrite(PIN_SENAL, LOW); pinMode(PIN_SENAL, OUTPUT);
  relojMotor = millis(); motor(false);
  pinMode(AZUL, OUTPUT); pinMode(VERDE, OUTPUT); pinMode(ROJO, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  Serial.begin(9600); Serial1.begin(BT_BAUD);
  Wire.begin(); Wire.setWireTimeout(25000, true);
  Wire.beginTransmission(LCD_ADDR); lcdOK = Wire.endTransmission() == 0;
  if (lcdOK) { lcd.init(); lcd.backlight(); }
  else decir("LCD no responde: revisar direccion I2C/cableado.");
  // Servos no reciben pulsos hasta verificar su alimentacion en Configuracion.h.
  if (SERVOS_VERIFICADOS) {
    s1.write(S1_NEUTRO); s2.write(S2_NEUTRO);
    s1.attach(S1_PIN); s2.attach(S2_PIN);
  }
  cargarEEPROM();
  bloqueadoBajo = analogRead(LDR_BAJO) < UMBRAL_BAJO;
  bloqueadoAlto = analogRead(LDR_ALTO) < UMBRAL_ALTO;
  presenciaCruda = presenciaEstable = analogRead(LDR_PRESENCIA) < UMBRAL_PRESENCIA;
  cambioPresencia = libreDesde = millis();
  if (!eepromFallo) mostrar("SmartSort GT", "Esperando cmd...");
  decir("SmartSort listo. Arranque siempre detenido.");
  if (!POTENCIA_VERIFICADA) decir("Motor bloqueado por configuracion; leer GUIA_CONEXIONES.md.");
}

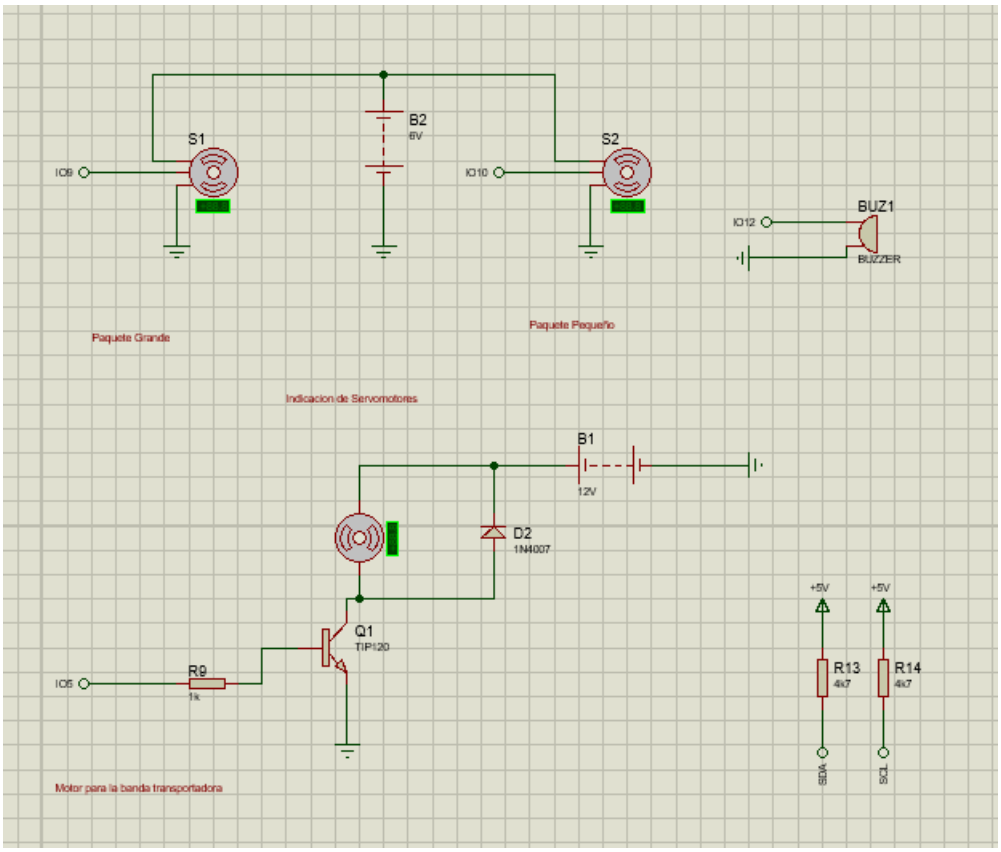
void loop() {
  actualizarProceso();
  recibir(Serial, usb); recibir(Serial1, bt);
  actualizarAvisos();
  if (monitorSensores && millis() - ultimoMonitor >= 750) {
    ultimoMonitor = millis(); mostrarSensores();
  }
}
```

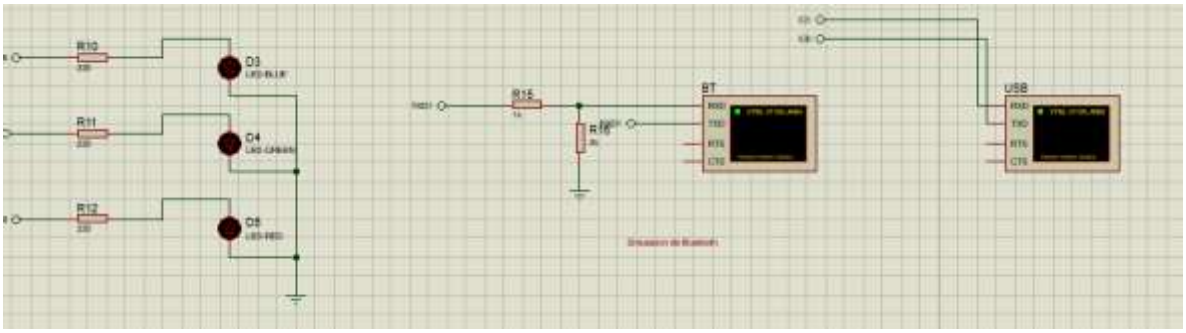
Proteus


LCD1
LM016L
VSS
VDD
VEE
RS
RW
E
D0
D1
D2
D3
D4
D5
D6
D7
GND
+5V
GND
U2
SCL
SDA
INT
A0
A1
A2
P0
P1
P2
P3
P4
P5
P6
P7
PCF8574
SCL
SDA
+5V
Pantalla LCD xd
S1
S2
B2
6V
IO9
IO10
IO12
BUZ1
BUZZER
Paquete Grande
Paquete Pequeño

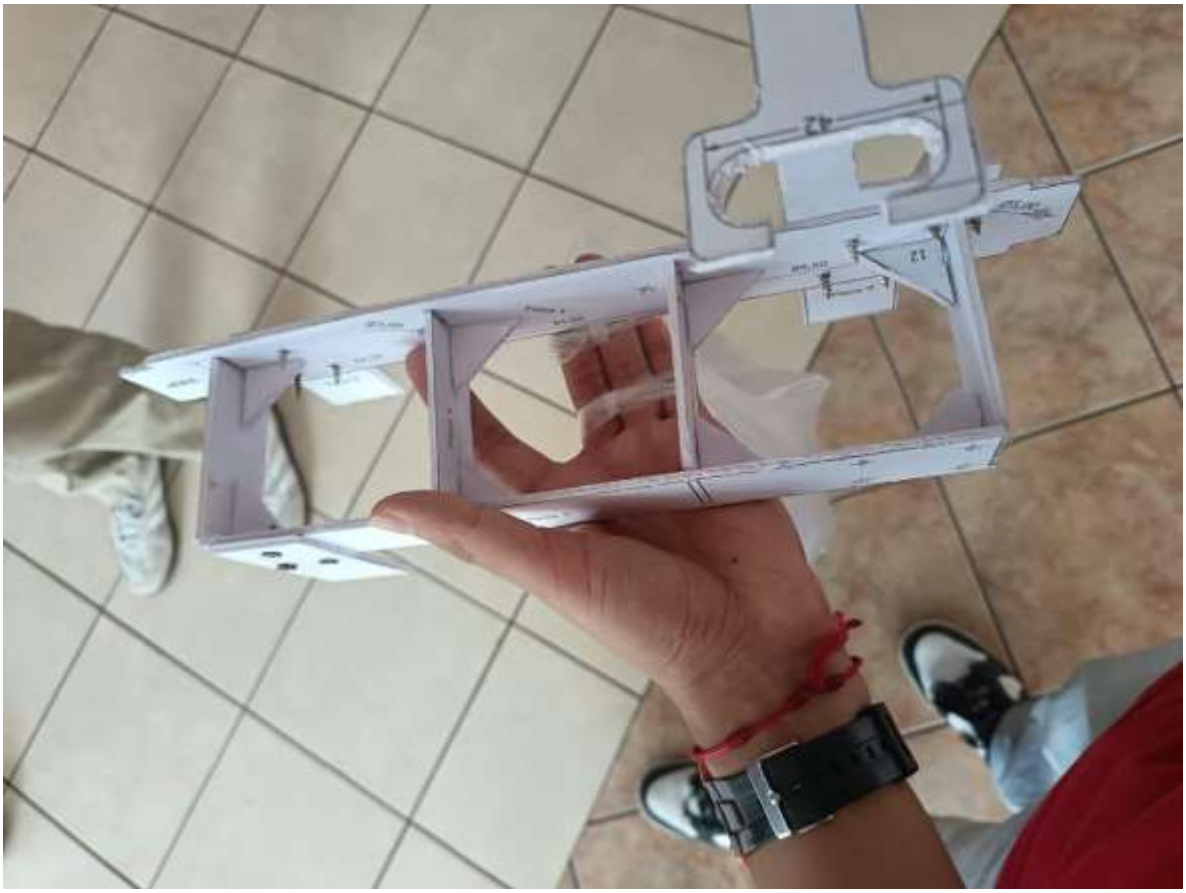

PROYECTO FINAL GRUPO7


RESET
XTAL1
XTAL2
PA0/AD0
PA1/AD1
PA2/AD2
PA3/AD3
PA4/AD4
PA5/AD5
PA6/AD6
PA7/AD7
PB0/SS/PCINT0
PB1/SCK/PCINT1
PB2/MOSI/PCINT2
PB3/MISO/PCINT3
PB4/OC2A/PCINT4
PB5/OC1A/PCINT5
PB6/OC1B/PCINT6
PB7/OC0A/OC1C/PCINT7
PC0/A8
PC1/A9
PC2/A10
PC3/A11
PC4/A12
PC5/A13
PC6/A14
PC7/A15
PD0/SCL/INT0
PD1/SDA/INT1
PD2/RXD1/INT2
PD3/TXD1/INT3
PD4/ICP1
PD5/XCK1
PD6/T1
PD7/T0
PE0/RXD0/PCINT8/PDI
PE1/TXD0/PDO
PE2/XCK0/AIN0
PE3/OC3A/AIN1
PE4/OC3B/INT4
PE5/OC3C/INT5
PE6/T3/INT6
PE7/ICP3/CLKO/INT7
AREF
AVCC
PF0/ADC0
PF1/ADC1
PF2/ADC2
PF3/ADC3
PF4/ADC4/TCK
PF5/ADC5/TMS
PF6/ADC6/TDO
PF7/ADC7/TDI
PG0/WR
PG1/RD
PG2/ALE
PG3/TOSC2
PG4/TOSC1
PG5/OC0B
PH0/RXD2
PH1/TXD2
PH2/XCK2
PH3/OC4A
PH4/OC4B
PH5/OC4C
PH6/OC2B
PH7/T4
PJ0/RXD3/PCINT9
PJ1/TXD3/PCINT10
PJ2/XCK3/PCINT11
PJ3/PCINT12
PJ4/PCINT13
PJ5/PCINT14
PJ6/PCINT15
PJ7
PK0/ADC8/PCINT16
PK1/ADC9/PCINT17
PK2/ADC10/PCINT18
PK3/ADC11/PCINT19
PK4/ADC12/PCINT20
PK5/ADC13/PCINT21
PK6/ADC14/PCINT22
PK7/ADC15/PCINT23
PL0/ICP4
PL1/ICP5
PL2/T5
PL3/OC5A
PL4/OC5B
PL5/OC5C
PL6
PL7
+5V
TXD
RXD
Arduino


S1
IO9
B2
6V
IO10
S2
IO12
BUZ1
BUZZER
Paquete Grande
Paquete Pequeño
Indicacion de Servomotores
B1
12V
D2
1N4007
Q1
TIP120
IO5
R9
1k
+5V
+5V
R13
4k7
R14
4k7
SDA
SCL
Motor para la banda transportadora


R10
D3
R11
D4
R12
D5
R15
BT
USB

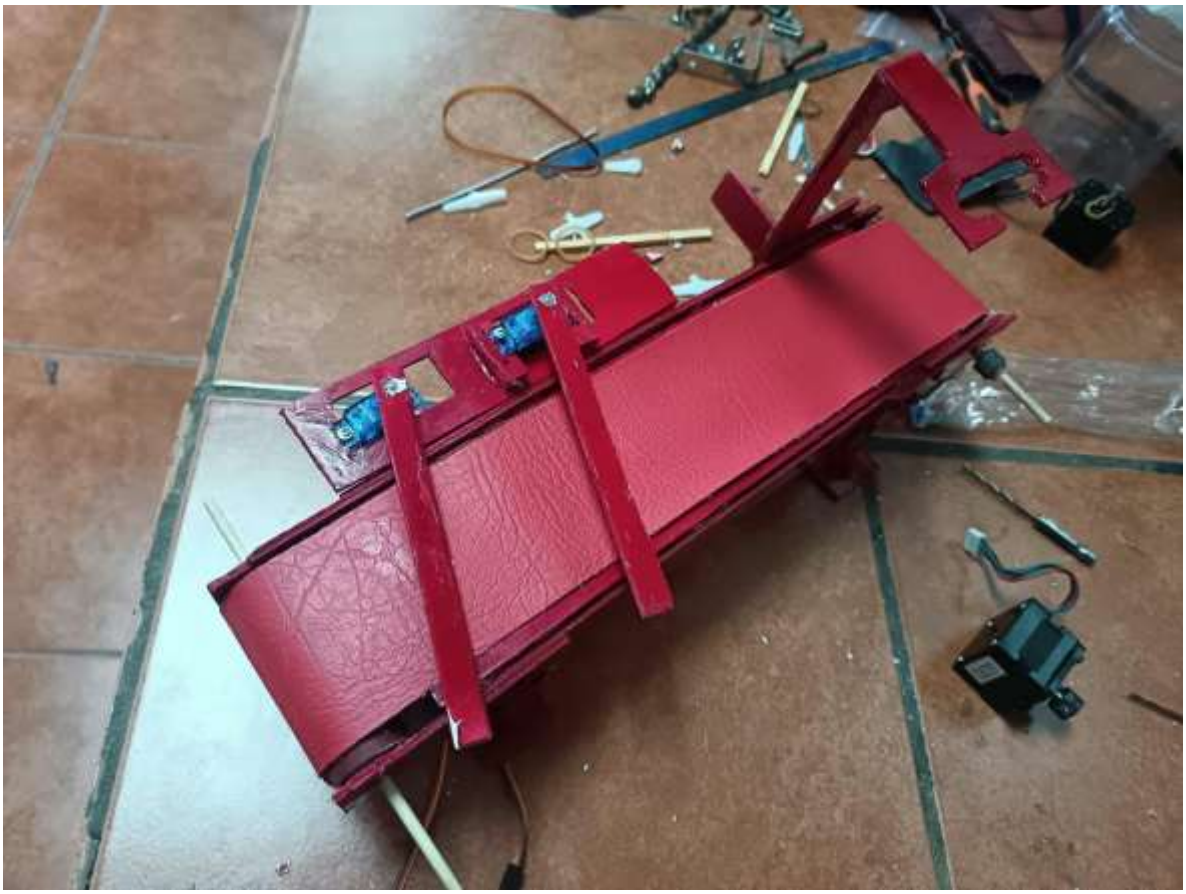
Maqueta







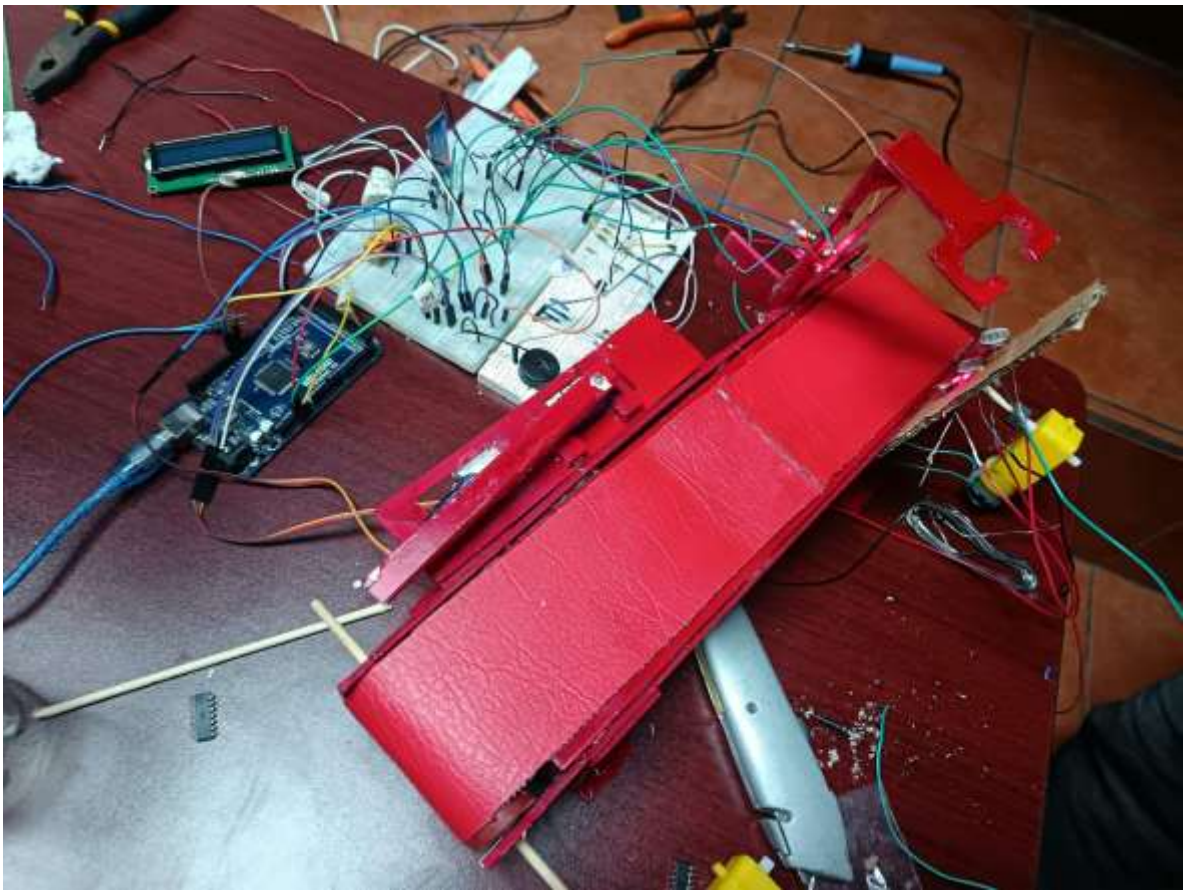














Presupuesto

Objeto	Unidades	Precio(Q)	Total(Q)
Arduino IDE	1	0.00	0.00
Proteus	1	0.00	0.00
LDR - Fotoresistensias	3	8.00	24.00
Diodo Laser	2	12.00	24.00
Buzzer	1	15.00	15.00
Motor Stepper*	1	0	0
Leds	3	6.00	18.00
Pantalla LCD2CC	1	60.00	60.00
Servomotores	2	31.00	62.00
Bluetooth HC-0605	1	59.00	59.00
Plancha PVC 3mm	1	80.00	80.00
Arcomate Vellum	6	10.00	10.00

Impresiones	-	-	12.00
Total			Q364

*El grupo ya contaba con Protoboards.

*El motor fue proporcionado sin costo.

Aportes

- **Brandon Otoniel Poitán Gálvez:**
  Colaboracion con armado de Protoboards
- **Luis Fernando Orive Perez:**
  Realizacion de Proteus y codigo Arduino.
- **Brandón Adán González Oliva:**
  Colaboracion con el armado de la maqueta.
- **Marcos Aarón Toledo Álvarez:**
  Compra de materiales y armado de la maqueta
- **Douglas Xavier Santiago Soto Mejia:**
  Documentacion y colaboracion con armado de maqueta

Equipo utilizado

Arduino Mega 
Pantalla LCD 
Servomotores 
Diodo Laser



Modulo Bluetooth



Plancha PVC



Resistencias



Cables Jumper Macho-Macho



Protoboard



LEDS



Conclusiones

-La implementación de SmartSort permitió integrar diferentes componentes electrónicos, como sensores láser, servomotores, Arduino y una banda transportadora, para automatizar el proceso de clasificación de paquetes según su tamaño.

-El proyecto permitió comprobar la importancia de la coordinación entre hardware y software, especialmente en la detección de los paquetes y el control de los servomotores, logrando un sistema más organizado y automatizado.

-El uso adecuado del Arduino mega demuestra que este puede ser de utilidad para poder automatizar ciertas tareas como en el caso de este proyecto que era la organización de paquetes por tamaño.

- El uso del módulo Bluetooth otorga mayor facilidad para ejecutar y controlar el proyecto permitiendo enviar comandos de manera remota y controlar las diferentes funciones del sistema de una forma más práctica y eficiente. Además, facilita las pruebas y la interacción con el prototipo, demostrando cómo la comunicación inalámbrica puede integrarse en sistemas automatizados.

.